Applied NLP

DATA 641

Lecture 7 — Introduction to convolutional neural networks

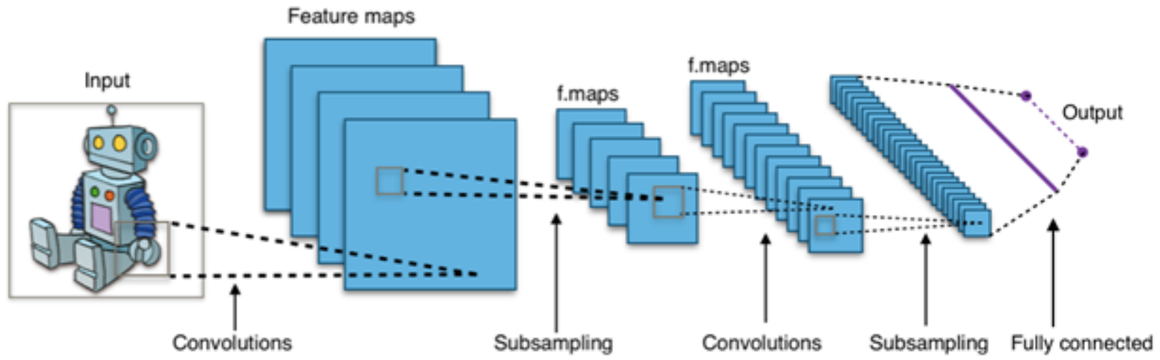
*Practical Natural Language Processing: A Comprehensive Guide to Building Real-World NLP Systems 1st Edition, Sowmya Vajjala, Bodhisattwa Majumder, Anuj Gupta, Harshit Surana, O' Reilly and Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 2nd Edition by Aurélien Géron

*Some figures are from the lecture notes: https://lena-voita.github.io/nlp_course; *Convolutional filter for images: https://github.com/vdumoulin/conv_arithmetic

Convolutional Neural Networks (CNN) for NLP

Class of deep neural networks most applied to analyzing visual imagery

- Image classification
- Facial recognition
- Object detection



Recently CNNs have been successfully used for several NLP tasks!!

Learning meaning—Why CNNs?

The nature of words are most tightly correlated to their relation to each other. That relationship can be expressed in at least two ways:

1. **Word order** here are two statements that do not mean the same thing:

`The dog chased the cat.` `The cat chased the dog.`

2. **Word proximity** here "shone" refers to the word "hull" at the other end of the sentence:

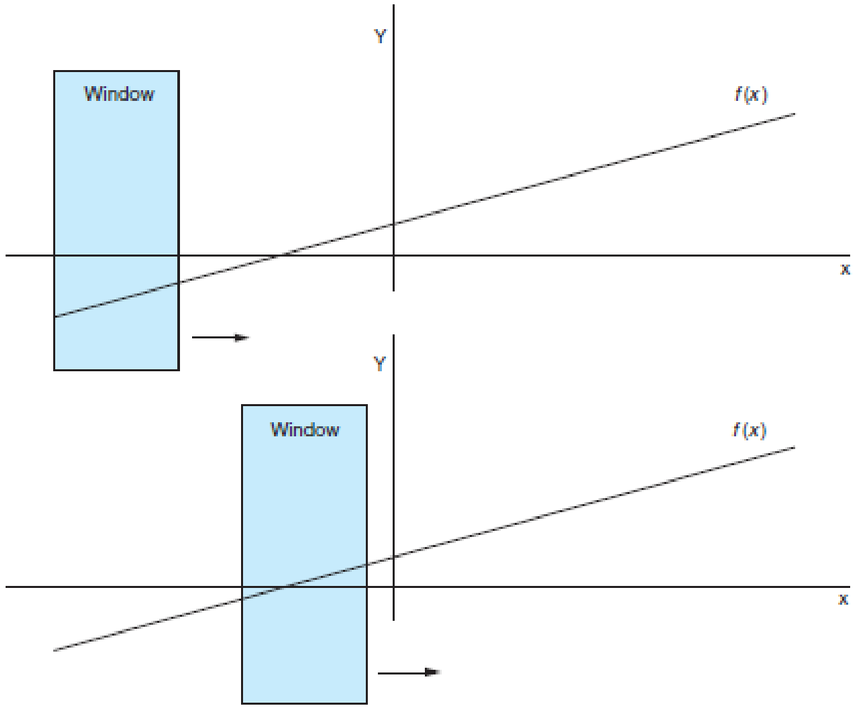
`The ship's hull, despite years at sea, millions of tons of cargo, and two mid-sea collisions, shone like new.`

These relationships can be explored for patterns in two ways: **spatially** and **temporarily**. CNNs will help with this issue.

Convolutional neural nets

Convolutional neural nets (CNNs), get their name from the concept of sliding (or convolving) a small window over the data sample.

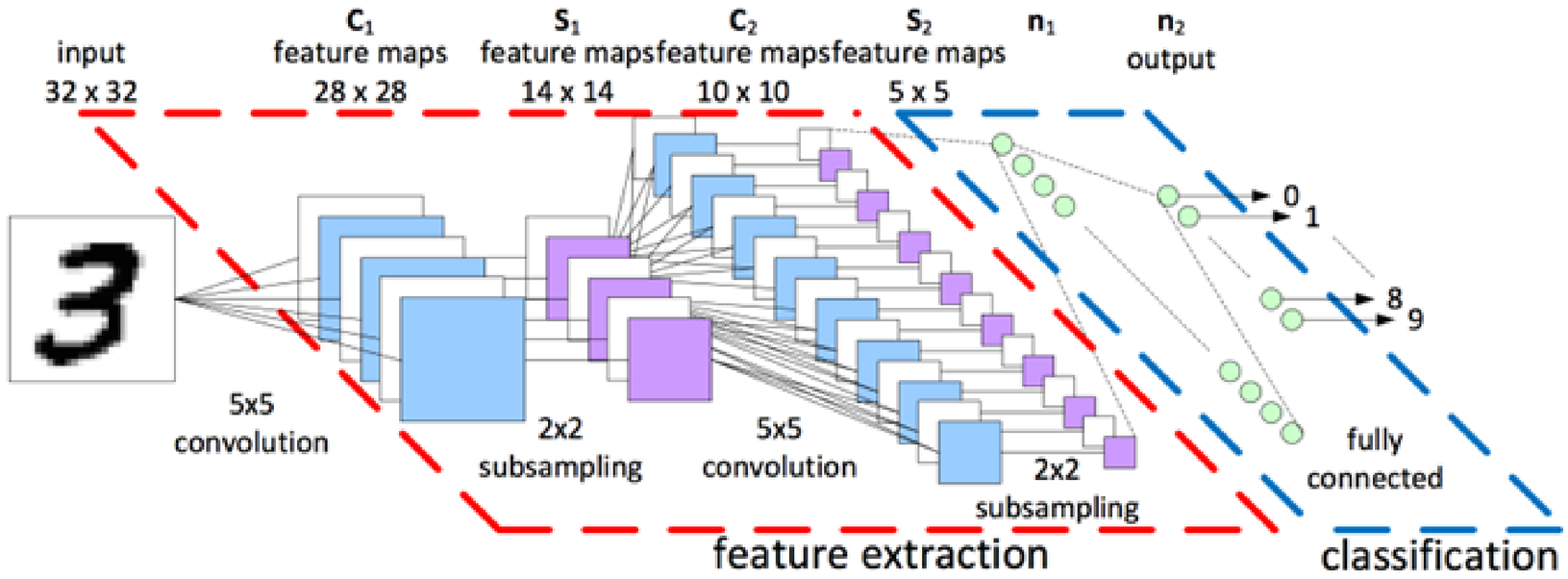
- Traditionally the window is used to slide over images to understand the concept and recently a window is used to slide over text. The idea behind this is that we are trying to look what can be seen through the window.



General architecture of CNN

The main components of a CNN are the following

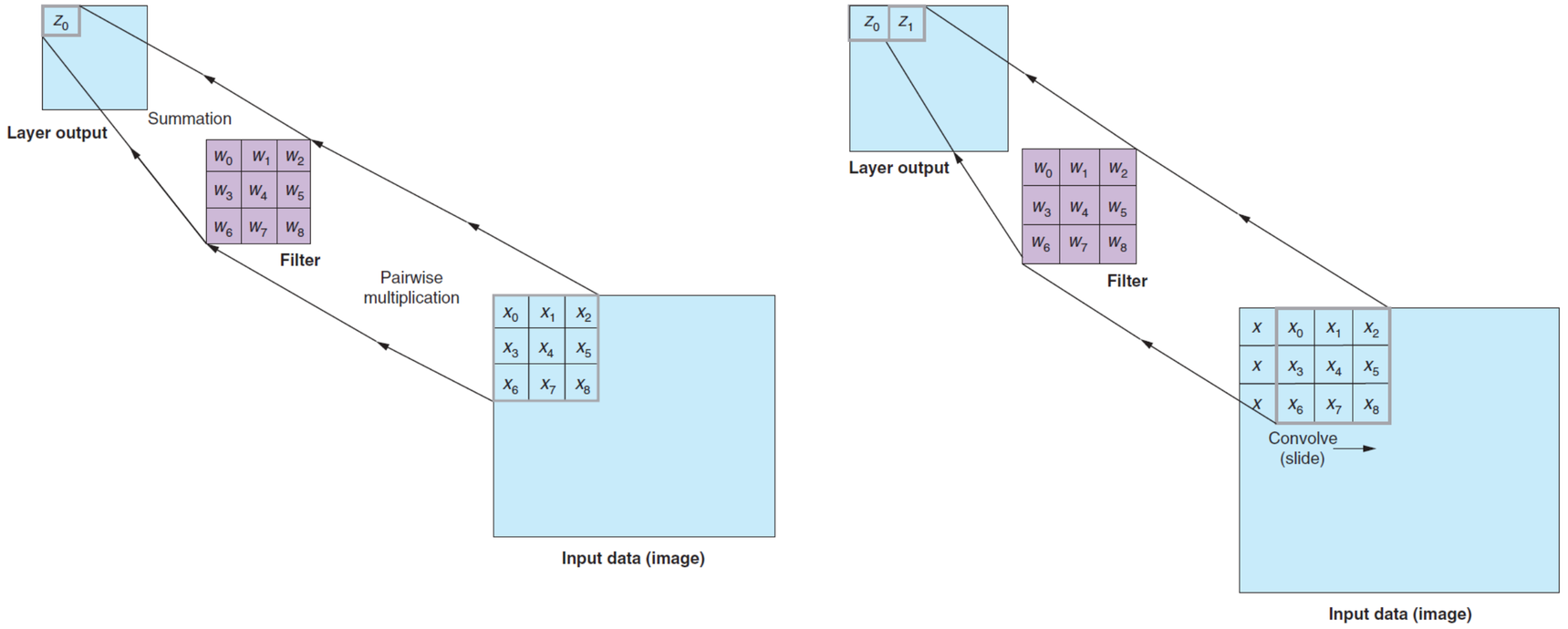
- Convolutional layers comprise neurons that scan their input for patterns
- Downsampling layers , or pooling layers are used to reduce the feature map for efficientcy



Filter composition

Filters are composed of two parts. A set of weights (exactly like the weights feeding into the neurons) and an activation function.

Filters are typically 3×3 (but often other sizes and shapes). Each filter in a convolutional neural net is unique, but each individual filter element is fixed within an image snapshot.

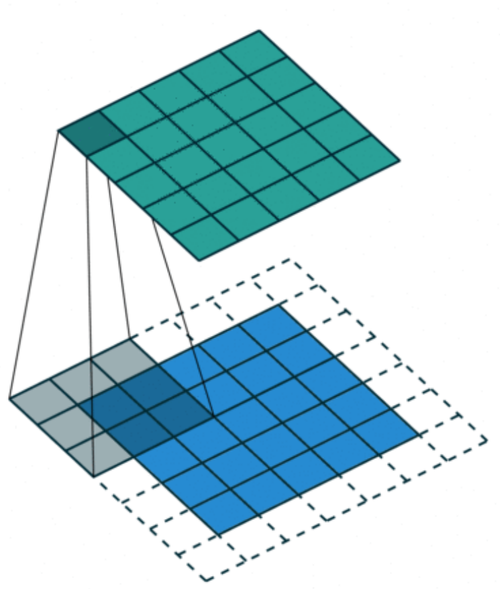


In the above figure, x_i is the value of the pixel at position i and z_0 is the output of a `ReLU` activation function (`z_0 = max(sum(x * w), 0)` or $z_0 = max(x_i * w_j), 0)$.

Filter composition in action

Convolutions in computer vision go over an image with a sliding window and apply the same operation, convolution filter, to each window.

Here the bottom is the input image, the top is the filter output.



Building blocks

- CNNs first came to prominence in image processing and image recognition.
    - In image recognition, the elements of each data point could be a 1 (on) or 0 (off) for each pixel in a black-and-white image
    - It could be the intensity of each pixel in a gray-scale image, or the intensity in each of the color channels of each pixel in a color image.
- Each filter you make is going to convolve or slide across the input sample
- How big are these filters we are talking about? The filter window size is a parameter to be chosen by the model builder and is highly dependent on the content of data.

Step size (stride)

- The distance traveled during the sliding phase is a parameter. And more importantly, it is almost never as large as the filter itself. Each snapshot usually has an overlap with its neighboring snapshot.

- The distance each convolution "travels" is known as the stride and is typically set to 1. Only moving one pixel (or anything less than the width of the filter) will create overlap in the various inputs to the filter from one position to the next. A larger stride that has no overlap between filter applications will lose the "blurring" effect of one pixel (or in your case, token) relating to its neighbors.

Padding

- If you start a 3×3 filter in the upper-left corner of an input image and stride one pixel at a time across, stopping when the rightmost edge of the filter reaches the rightmost edge of the input, the output "image" will be two pixels narrower than the source input.

- Keras has tools to help deal with this issue. The first is to ignore that the output is slightly smaller. The Keras argument for this is `padding='valid'`.

- The downfall of this strategy is that the data in the edge of the original input is under-sampled as the interior data points are passed into each filter multiple times, from the overlapped filter positions.

- On a large image, this may not be an issue, but as soon as you bring this concept to bear on a Tweet, for example, under-sampling a word at the beginning of a 10-word dataset could drastically change the outcome.

- The next strategy is known as padding, which consists of adding enough data to the input's outer edges so that the first real data point is treated just as the innermost data points are. The downfall of this strategy is that you are adding potentially unrelated data to the input, which in itself can skew the outcome. You don't care to find patterns in fake data that you generated after all. But you can pad the input several ways to try to minimize the ill effects. See the following listing.

Original array:

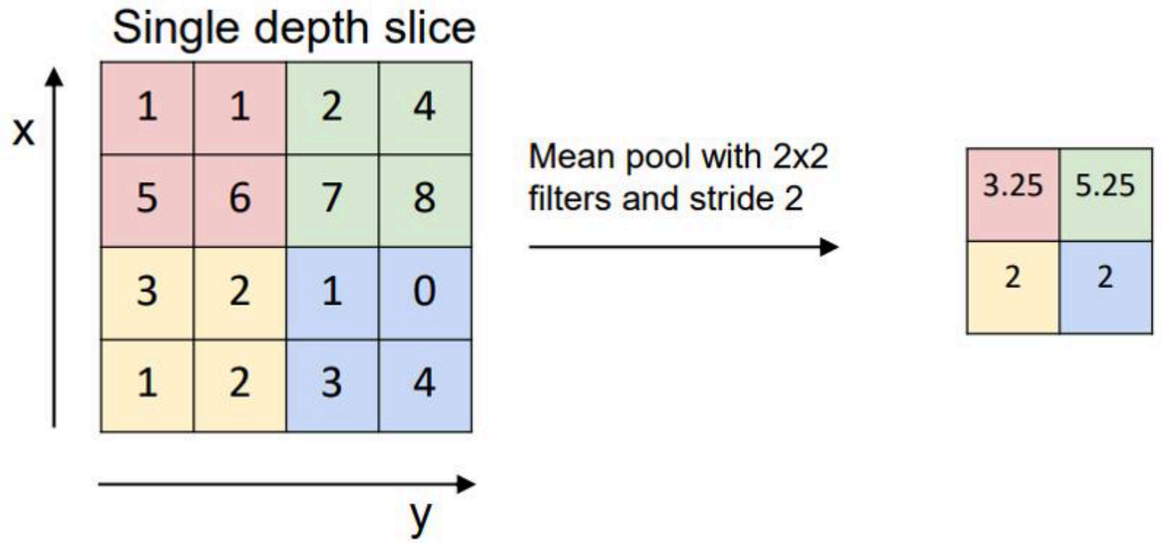
$$\begin{bmatrix} * & * & * & * & * \\ * & * & * & * & * \\ * & * & * & * & * \\ * & * & * & * & * \\ * & * & * & * & * \end{bmatrix}$$

After padding:

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & * & * & * & * & * & 0 & 0 \\ 0 & 0 & * & * & * & * & * & 0 & 0 \\ 0 & 0 & * & * & * & * & * & 0 & 0 \\ 0 & 0 & * & * & * & * & * & 0 & 0 \\ 0 & 0 & * & * & * & * & * & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Pooling

Spatial pooling or Down sampling reduces the dimensionality of each map but retains the important information

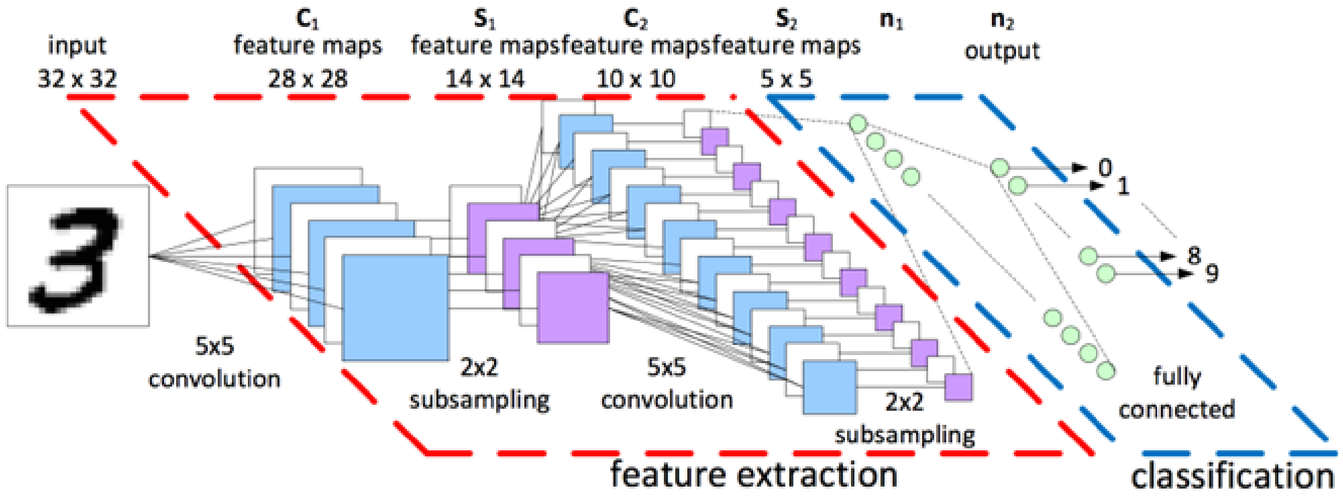


Finally, we vectorize our matrix and feed it into a fully connected layer

Convolutional pipeline

- You have n filters and n new images now. What do you do with that? This, like most applications of neural networks, starts from the same place: a labeled dataset.

- And likewise you have a similar goal. To predict a label given a novel image. The simplest next step is to take each of those filtered images and string them out as input to a feed-forward layer.



- No matter how many layers (convolutional or otherwise) you add to your network, once you have a final output you can compute the error and backpropagate that error all the way back through the network.